

Inventory Management System

Case 1

Presenter : **Baimagambet Gauhar**
Mukazhan Mumina
Mukhtarova Dana
Nurzhan Aisu

Problem description

01

Main problem

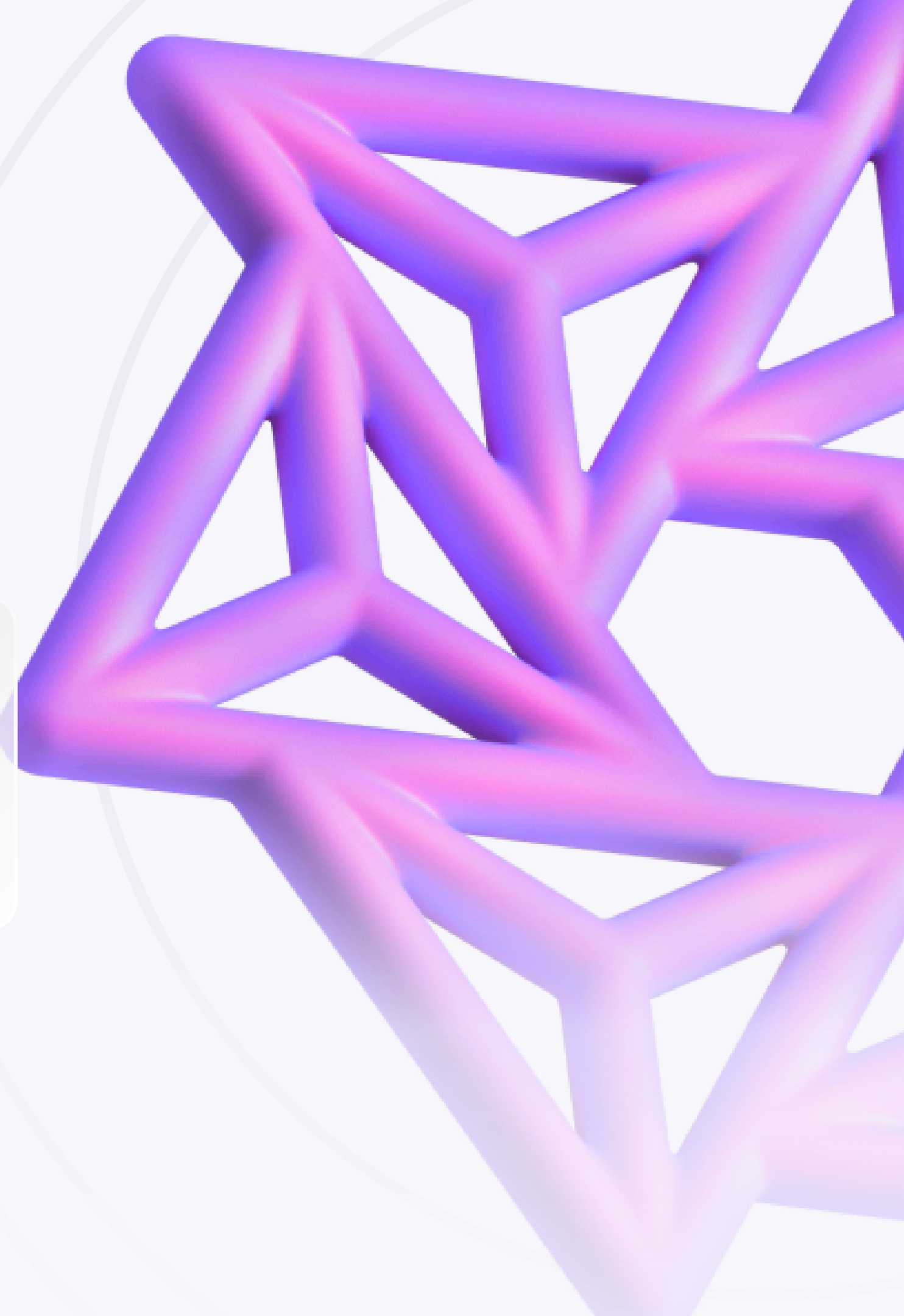
No centralized product tracking
Manual stock counting — error prone
Expired items go unnoticed
No low-stock alerts or reporting
Data lost between sessions

02

Project goal

CLI app with full CRUD operations
JSON persistence across sessions
Expiry date tracking & alerts
Low stock detection & reporting
CSV export for external use

Target users: small warehouses, storerooms, office supply managers



System Design & Architecture

models/product.py

Product
DiscountedProduct

services/inventory.py

InventoryService
ProductNotFoundError

services/report.py

ReportService
Summary & Expiry

utils/validators.py

JSON · CSV · Input
Validation · Decorators

Key decision — why dict, not list?

O(1) lookup by ID instead of O(n) scan — critical for large catalogs.

```
class InventoryService: 6+ usages
    def __init__(self):
        self._products = {}
        self._categories = set()
        self._next_id = 1
```

Data Flow

main.py

InventoryService

FileHandler

inventory.json / CSV

OOP Design

Class Hierarchy

Product — base class with @property for price, quantity, name

DiscountedProduct extends Product — adds discount field

Setters validate data on every assignment

to_dict / from_dict — JSON serialization

__eq__, __hash__ — comparison by product_id

Why @property?

Client writes p.price = -5 — gets ValueError immediately. Data stays valid at all times.

Code from product.py:

@property — validation on every assignment

```
@property 10 usages (6 dynamic)  🧑 gauharb
def price(self):
    return self._price

@price.setter 7 usages (6 dynamic)  🧑 gauharb
def price(self, value :{__lt__}):
    if value < 0:
        raise ValueError("Price cannot be negative")
    self._price = value
```

Inheritance — DiscountedProduct extends Product

```
class DiscountedProduct(Product): 9+ usages  🧑 gauharb
    """Discounted product - inherited from Product"""
```

```
def to_dict(self): 3 usages (2 dynamic)  🧑 gauharb
    d = super().to_dict()
    d["discount_percent"] = self._discount
    return d
```

OOP Concepts in Action

Encapsulation — Product uses private attributes and @property methods.

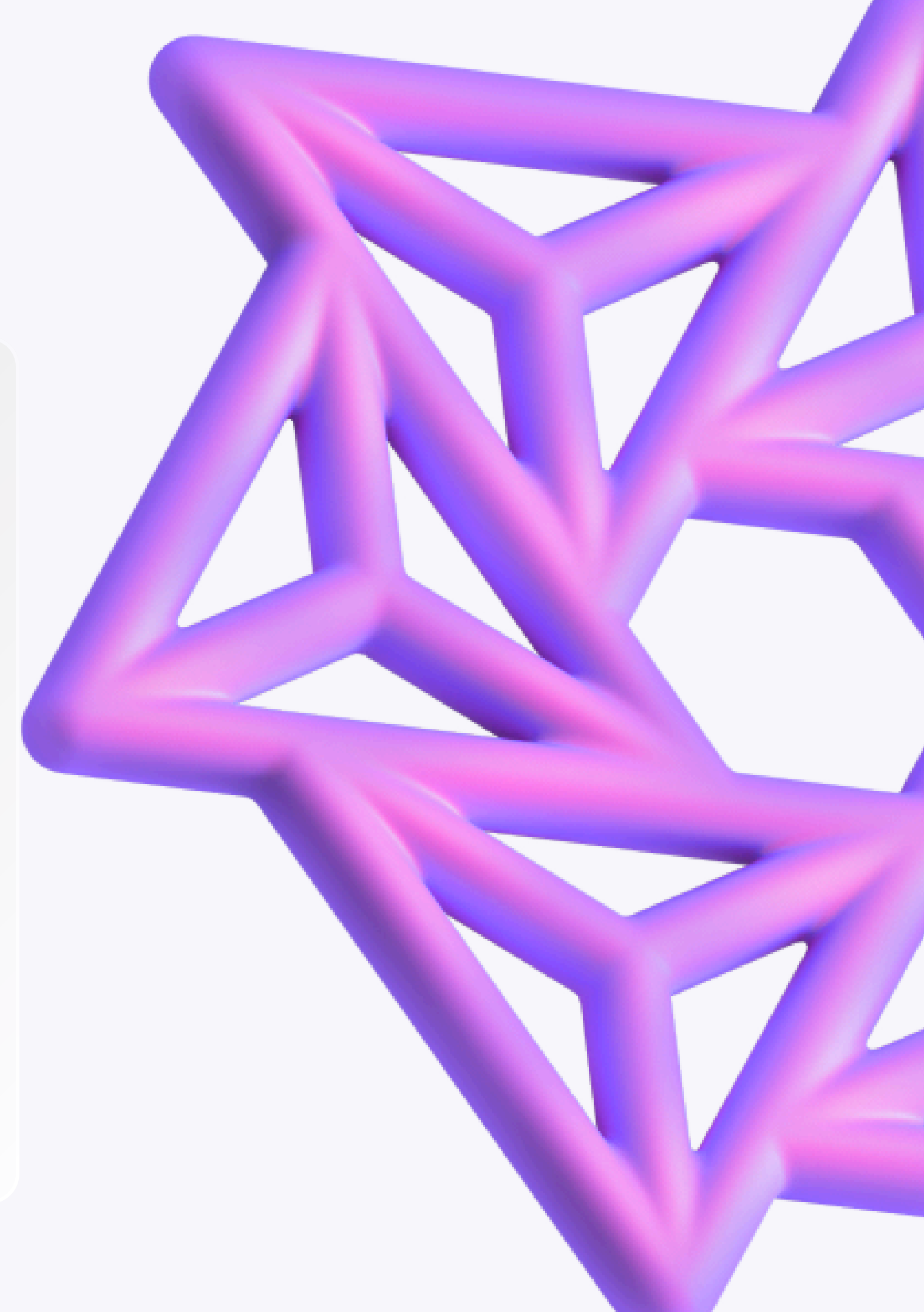
Inheritance — DiscountedProduct extends Product.

Polymorphism — final_price() works differently for regular and discounted products.

Error Handling — custom exceptions make the system safer.

File Handling — JSON and CSV are used for saving and exporting data.

Unit Testing — unittest checks system reliability.



Complex Parts — Decorators

What is a decorator?

A wrapper function that adds behaviour to another function without modifying its code.

@log_action

Prints the function name before and after each call.
Applied to do_add, do_remove, do_restock, do_sell in main.py.

@validate_positive

Checks amount > 0 before restock and sell. One decorator protects two methods. DRY principle.

Without decorators we would copy the check into every function manually.

utils/validators.py + services/inventory.py

validators.py

```
def log_action(func): 6+ usages @ mukhtrvaad *
    @functools.wraps(func) @ mukhtrvaad
    def wrapper(*args, **kwargs):
        print(f" [LOG] Executing: {func.__name__}()")
        result = func(*args, **kwargs)
        print(f" [LOG] Done: {func.__name__}()")
        return result
    return wrapper
```

inventory.py — DRY: one decorator, two methods

```
@validate_positive @ gauharb *
def restock_product(self, product_id, amount):
    product = self._get_or_raise(product_id)
    product.quantity += amount
    return product

@validate_positive 3 usages (1 dynamic) @ gauharb
def sell(self, product_id, amount):
    """Sell product """
    product = self._get_or_raise(product_id)

    if product.quantity < amount:
        raise ValueError(f"Not enough stock: {product.quantity} available")

    product.quantity -= amount
    return product
```

main.py — logging without touching do_add

```
@log_action 1 usage @ aisunurzhan08-collab
def do_add(service : {add_product, __eq__} ):
    print("\n Add new product ")
```

Complex Parts — Functional Style & Expiry

filter() + lambda + map()

Python functional style — built-in functions instead of for-loops.

- filter(lambda, items) — filters by condition
- map(lambda, items) — applies function to each element
- Used in 6+ places: expiry, low stock, total value

Expiry Date Logic

- date.fromisoformat() parses string '2026-05-20'
- days_until_expiry() returns int or None
- None means no date set (not an error!)

services/inventory.py + models/product.py

filter + lambda — expiring products

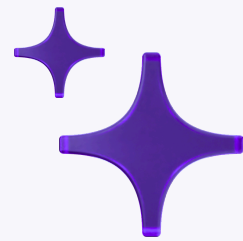
```
def get_expiring_soon(self, days :int =7): 1 usage  👤 gauharb
    """
    Get products expiring within the next N days
    Uses filter + lambda
    """

    return list(filter(
        lambda p : {days_until_expiry} : p.days_until_expiry() is not None
        and 0 <= p.days_until_expiry() <= days,
        self._products.values()
    ))
```

product.py — expiry date logic

```
def is_expired(self): 8 usages (5 dynamic)  👤 gauharb
    """Returns True if the expiration date has passed"""
    if self._expiry_date is None:
        return False
    expiry = date.fromisoformat(self._expiry_date)
    return expiry < date.today()
```

Optimization



dict vs list

O(1) ID lookup
vs O(n) scan

set for categories

O(1) membership
checks

Generator

iter_products() — yield
doesn't load all to RAM

Lazy filter()

Computed only
when needed

services/inventory.py — generator + auto-ID

Generator — memory efficient

```
def iter_products(self): 1 usage  @gauharb
    """Generator that yields products one-by-one"""

    for product in self._products.values():
        yield product
```

Auto-increment ID — O(1)

```
self._next_id += 1
```

```
self._next_id = max(self._products.keys()) + 1
```

DRY: one decorator for two methods

```
@validate_positive @gauharb *
def restock_product(self, product_id, amount):
    product = self._get_or_raise(product_id)
    product.quantity += amount
    return product

@validate_positive 3 usages (1 dynamic) @gauharb
def sell(self, product_id, amount):
    """Sell product """
    product = self._get_or_raise(product_id)

    if product.quantity < amount:
        raise ValueError(f"Not enough stock: {product.quantity} available")

    product.quantity -= amount
    return product
```

Key Data Structures ✨

Dictionary `self._products` — stores products by ID and gives $O(1)$ lookup.

Set `self._categories` — stores unique categories without duplicates.

List — used for search and filter results.

Generator `yield` — iterates products one by one without loading everything into memory.

Demo — Program Features

```
MENU = """  
  
    INVENTORY MANAGEMENT SYSTEM  
  
1.  All products  
2.  Add new Product  
3.  Remove Product  
4.  Update Product  
5.  Search by Name  
6.  Filter by Category  
7.  Low Stock  
8.  Summary Statistics  
9.  Restock  
10. Sell Product  
11. Expiry Report  
12. Export to CSV  
0.  Save and Exit  
  
"""
```

Add Product (option 2)

Accepts name, price, qty, category, expiry + discount. All validated via prompt_*

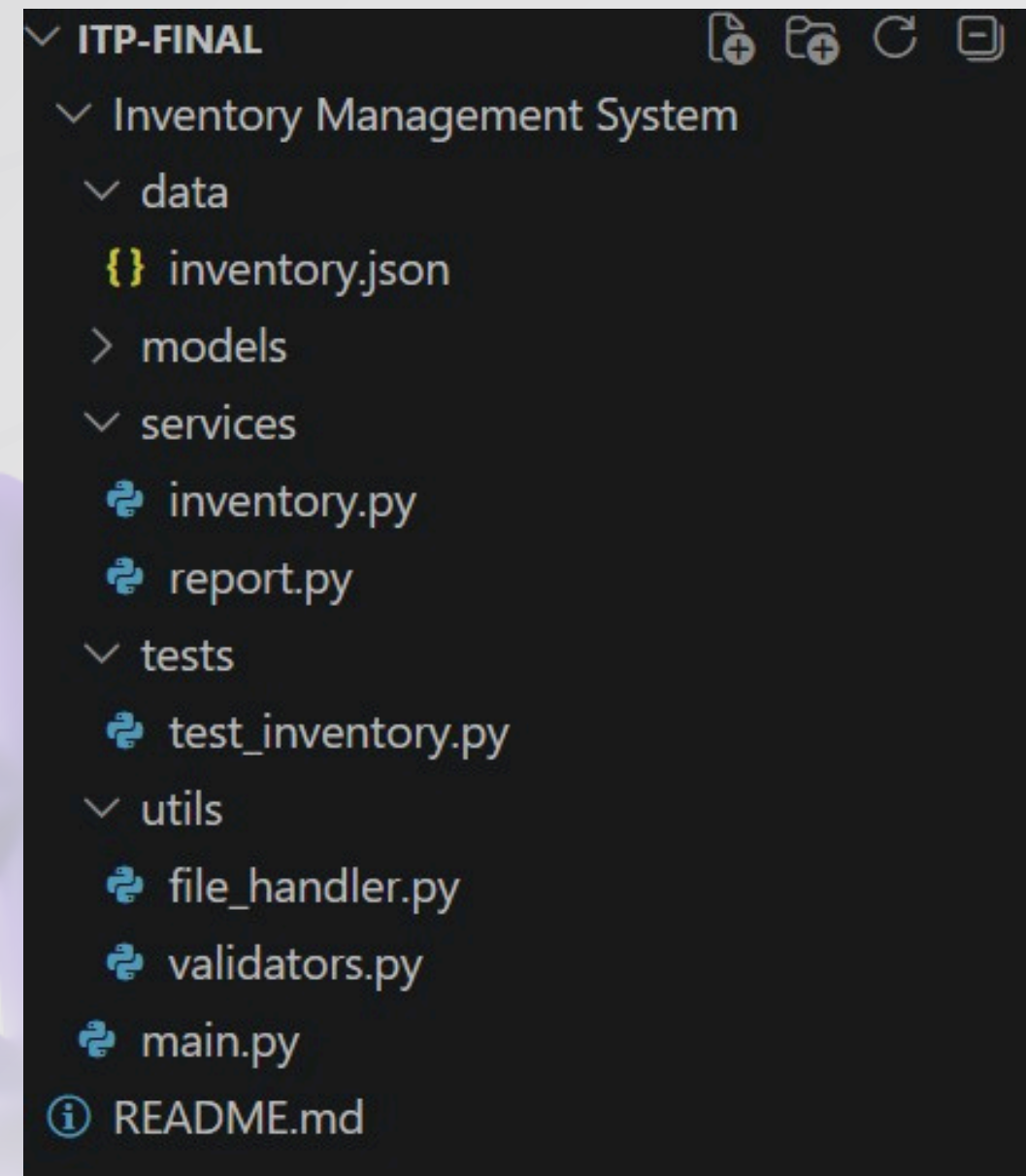
```
@log_action 1 usage  @ aisunurzhan08-collab  
def do_add(service : {add_product, __eq__} ):  
    print("\n Add new product ")
```

Sell Product (option 10)

Checks remaining stock — raises ValueError if insufficient. Decrements quantity.

```
def sell(self, product_id, amount):  
    """Sell product """  
    product = self._get_or_raise(product_id)  
  
    if product.quantity < amount:  
        raise ValueError(f"Not enough stock: {product.quantity} available")  
  
    product.quantity -= amount  
    return product
```

Demo — Program Features



CLI Menu

This menu is implemented in main.py. It allows the user to interact with the system. Each menu option is connected to a specific method from the service classes.

Demo — Sample Inventory Data

ID	Name	Price (\$)	Quantity	Category	Description	Expiry Date	Discount (%)
1	Laptop	1 270,00	5	electronics	15-inch business laptop		
2	Mouse	25,00	50	electronics	Wireless optical mouse		
3	Milk	1,50	10	food	Fresh whole milk	2026-12-25	
4	Monitor	350,00	8	electronics	27-inch 4K monitor		10,0%
5	Yogurt	2,00	15	food	Strawberry yogurt	2026-05-20	
6	Webcam	89,99	2	electronics	1080p webcam		
7	Cheese	5,00	8	food	Cheddar cheese	2026-05-23	
8	Notebook	4,50	200	stationery	A5 ruled notebook		
9	HDMI Cable	12,00	0	accessories	2m HDMI cable		
10	Standing Desk	499,00	1	furniture	Electric height-adjustable desk		15,0%
11	Keyboard	79,99	30	electronics	Mechanical RGB keyboard		
12	USB Hub	35,00	20	accessories	7-port USB 3.0 hub		
13	Butter	3,20	25	food	Unsalted butter 200g	2026-06-15	
14	Office Chair	320,00	4	furniture	Ergonomic mesh chair		5,0%
15	Pen Pack	2,99	500	stationery	Blue ballpoint pens 10pcs		
16	Headphones	149,00	12	electronics	Noise-cancelling over-ear		
17	Bread	1,80	20	food	Whole wheat loaf	2026-05-22	
18	Desk Lamp	45,00	15	furniture	LED adjustable desk lamp		
19	Ethernet Cable	8,50	60	accessories	Cat6 5m patch cable		

⚠ = low stock / expiring soon × = out of stock % = discounted | 19 products total in inventory.json

Tests — test_inventory.py

30+ unit tests — what do we check?

- *Product*: $price < 0 \rightarrow ValueError$
- *DiscountedProduct*: discount 0–100
- *Expiry*: `is_expired()`, `days_until_expiry()`
- *InventoryService*: `add`, `remove`, `update`, `search`
- `sell()`: no stock $\rightarrow ValueError$
- `filter_by_price_range`: $min > max \rightarrow ValueError$
- `update_product`: unknown field $\rightarrow ValueError$

Run: `python -m pytest tests/ -v`

tests/test_inventory.py

```
def test_sell_not_enough_stock(self):  @ muminamukazhan
    p = self._add_laptop()
    with self.assertRaises(ValueError):
        self.service.sell(p.product_id, amount=100)
```

```
def test_days_until_expiry(self):  @ muminamukazhan
    future = (date.today() + timedelta(days=10)).isoformat()
    p = Product(product_id=4, name="Cheese", price=3.0, quantity=2, expiry_date=future)
    self.assertEqual(p.days_until_expiry(), second=10)
```

```
def test_update_unknown_field(self):  @ muminamukazhan
    svc = InventoryService()
    p = svc.add_product(name="Test", price=10.0, quantity=1)
    with self.assertRaises(ValueError):
        svc.update_product(p.product_id, color="red")
```

```
def test_negative_price_raises_error(self):  @ muminamukazhan
    with self.assertRaises(ValueError):
        Product(product_id=2, name="X", price=-1.0, quantity=0)
```

Team Contribution Breakdown

Baimagambet Gauhar

Product + Inventory

- Product, DiscountedProduct classes
- @property + setters with validation
- to_dict / from_dict (JSON I/O)
- InventoryService — full CRUD
- Generator iter_products()

Mukhtarova Dana

Report + Validators

- ReportService — all print methods
- Summary, category breakdown
- Expiry report logic
- @log_action, @validate_positive
- prompt_* — user input helpers

Nurzhan Aisu

File Handler + Main

- FileHandler — JSON load/save
- CSV export (full, low-stock, expired)
- main.py — menu and handlers
- Integration of all modules
- do_add, do_sell, do_update...

Mukazhan Mumina

Tests + Database + README

- test_inventory.py — 36 tests
- TestProduct, TestInventoryService
- TestExpiry, TestEdgeCases
- inventory.json — 19 products
- README and documentation

Thank You!

19

Products
in sample data

12

Menu
operations

36

Unit
tests

4

Modules
(clear SoC)

Built with: Python 3 · OOP · Decorators · Generators · JSON · CSV · unittest

Inventory Management System